

Buổi 2

Tổng tiền tố, hai con trỏ và cửa sổ trượt, bản Python

Kiến thức, mười bài tập và mười code mẫu

Ngày 23 tháng 9 năm 2026

TS. Đỗ Phúc Hảo

Tài liệu một file cho buổi 2, bản Python. Gồm bốn phần: kiến thức, mười đề bài, gợi ý và lời giải, và mười code mẫu đầy đủ.

Đề bài và bộ test dùng chung với bản C++ ở `../../topics/02-prefix-twopointer/`. Cùng một đề, cùng một bộ test, khác ngôn ngữ, nên bạn so sánh được hai lối viết cạnh nhau. Cái khác không nằm ở thuật toán mà ở **hình dạng code mà ngôn ngữ chịu được**, và mục I.8 là chỗ nói thẳng chuyện đó bằng số đo trên cả mười bài.

1. Mục lục

Phần	Nội dung
I	Kiến thức, mười hai mục
II	Mười đề bài A tới J
III	Gợi ý phân tầng và lời giải từng bài
IV	Mười code mẫu đầy đủ

Phần I. Kiến thức

Mục tiêu: nhìn thấy chữ ”đoạn con liên tiếp” trong đề là biết ngay nên đi hướng nào. Ba kỹ thuật của buổi này phủ một tỷ lệ rất lớn các bài dễ và trung bình của ICPC, và quan trọng hơn, chúng là nền của segment tree lẫn quy hoạch động về sau.

Mục	Nội dung	Bài luyện
1	Tổng tiền tố một chiều	A, B
2	Mảng hiệu, cộng cả đoạn	C
3	Tổng tiền tố hai chiều	D
4	Hai con trỏ	E, I
5	Cửa sổ trượt và điều kiện đơn điệu	F, G
6	Khi hai con trỏ không dùng được	H
7	Hàng đợi đơn điệu	J
8	Đo thật, mười bài cạnh nhau	tất cả
9	Đọc dữ liệu và ghi kết quả	tất cả
10	Cạm bẫy riêng của Python	tất cả
11	Những cạm bẫy của bản C++ đã biến mất	tất cả
12	Bộ khung tự hỏi khi gặp đề mới	tất cả

1. Tổng tiền tố một chiều

Ý tưởng duy nhất: đổi một phép tính lặp lại nhiều lần thành một bảng tra.

```
from itertools import accumulate

p = [0] * (n + 1)
p[1:] = accumulate(a)          # a là danh sách n phần tử, danh số tu 0
# tổng của a[l..r] với chỉ số danh tu 1
s = p[r] - p[l - 1]
```

Xây bảng tốn $O(n)$, mỗi truy vấn sau đó tốn $O(1)$. Với q truy vấn thì tổng là $O(n + q)$ thay vì $O(n \cdot q)$.

Vì sao luôn để mảng dài $n + 1$ và đánh số từ 1. Vì công thức cần $p[l - 1]$, mà với $l = 1$ thì đó là $p[0]$. Nếu bạn đánh số từ 0 thì phải viết một câu điều kiện cho trường hợp $l = 0$, tức là một chỗ dễ quên, lặp lại ở mọi truy vấn. Thêm một ô trống ở đầu là cái giá rẻ nhất bạn từng trả.

Và với Python có một lý do thứ hai, nặng hơn: $p[-1]$ **không** báo lỗi, nó lấy phần tử cuối mảng. Một lỗi lệch chỉ số trong C++ có cơ hội nổ ra thành lỗi truy cập; trong Python nó lặng lẽ trả về một con số sai. Đánh số từ 1 là cách duy nhất làm cho chỉ số âm không bao giờ xuất hiện.

Kiểu dữ liệu. Tổng của $2 \cdot 10^5$ số cỡ 10^9 là $2 \cdot 10^{14}$. Bản C++ bắt buộc phải khai long long. Trong Python số nguyên lớn bao nhiêu cũng được, nên cạm bẫy này **không tồn tại**. Đây là cạm bẫy đầu tiên trong tám cạm bẫy của bản C++ biến mất ở đây; xem mục 11 để biết danh sách đủ.

Dùng accumulate thay cho vòng lặp. `itertools.accumulate` chạy vòng cộng bên trong C thay vì trong trình thông dịch. Đo trên bộ test lớn nhất của bài A, bản dùng `accumulate` mất 82 ms còn bản viết vòng for bằng Python mất 91 ms. Chênh lệch nhỏ vì phần nặng của bài A nằm ở chỗ khác, nhưng thói quen này sẽ đáng giá hơn nhiều ở bài D.

Một biến thể hay gặp: chia dãy thành hai phần tại vị trí i . Tổng bên trái là $p[i - 1]$, tổng bên phải là $total - p[i]$. Đó là toàn bộ bài B, và ở bài B thì không cần cả mảng tiền tố, chỉ cần một biến cộng dồn.

2. Mảng hiệu, cộng cả đoạn

Đây là tổng tiền tố đọc ngược. Bài toán: có q thao tác "cộng v vào mọi phần tử từ l đến r ", cuối cùng in mảng. Làm thẳng là $O(n \cdot q)$, tức $4 \cdot 10^{10}$ phép với giới hạn thường gặp.

```
d = [0] * (n + 2)
# cộng v vào a[l..r]
d[l] += v
d[r + 1] -= v
# khôi phục mảng bằng một lần tổng tiền tố
a = list(accumulate(d[1:n + 1]))
```

Mỗi thao tác chỉ đụng vào hai ô, nên tổng chi phí là $O(n + q)$.

Hiểu bản chất: d là mảng hiệu, $d[i] = a[i] - a[i-1]$. Cộng v vào cả đoạn thì mọi hiệu ở giữa đoạn không đổi, chỉ có hiệu ở mép trái tăng v và hiệu ngay sau mép phải giảm v . Lấy tổng tiền tố của mảng hiệu thì ra lại mảng gốc.

Vì sao khai $n + 2$ chứ không phải $n + 1$. Vì khi r bằng n thì $d[r + 1]$ là $d[n + 1]$.

Và đây là chỗ hai ngôn ngữ xử sự khác hẳn nhau trước cùng một lỗi:

Ngôn ngữ	Chuyện gì xảy ra khi thiếu một ô
C++, vector dài $n + 1$	ghi ra ngoài vùng, không báo gì, âm thầm đề lên bộ nhớ khác
Python, list dài $n + 1$	<code>IndexError</code> ngay tại dòng đó, bộ chấm kết luận RE

Hai chuyện này nghe như Python tốt hơn, và trong trường hợp này đúng là thế. Nhưng đừng rút ra bài học "Python sẽ báo cho tôi biết": chỉ số **âm** thì Python không báo gì cả, nó lấy phần tử cuối. Ranh giới là: vượt quá bên phải thì kêu, vượt quá bên trái thì im.

Kỹ thuật này còn có tên là **imos**, và phiên bản hai chiều của nó là cách tô màu cả hình chữ nhật trong $O(1)$.

3. Tổng tiền tố hai chiều

```
# p[i][j] = tổng của luoi con tu (1,1) tới (i,j)
p[i][j] = a[i][j] + p[i-1][j] + p[i][j-1] - p[i-1][j-1]
# tổng hình chữ nhật (r1,c1) tới (r2,c2)
s = p[r2][c2] - p[r1-1][c2] - p[r2][c1-1] + p[r1-1][c1-1]
```

Cả hai công thức đều là nguyên lý bù trừ. Khi xây bảng, hai hình chữ nhật con chồng lên nhau ở góc trên trái nên phần chồng bị cộng hai lần, phải trừ ra một lần. Khi truy vấn thì ngược lại, hai dải bị trừ chồng nhau ở một góc nên góc đó bị trừ hai lần và phải cộng lại một lần. **Số hạng góc mang dấu ngược nhau ở hai công thức**, và đó chính là chỗ hay nhớ nhầm. Cách nhớ chắc nhất là vẽ bốn hình chữ nhật ra giấy chứ đừng học thuộc đầu.

Cạm bẫy chỉ Python mới có, và nó ở ngay dòng cấp phát bảng.

```
p = [[0] * (m + 1)] * (n + 1) # SAI: n+1 tham chiếu tới MỘT hàng duy nhất
p = [[0] * (m + 1) for _ in range(n + 1)] # đúng: n+1 hàng riêng biệt
```

Dòng thứ nhất nhân một **list** lên $n + 1$ lần, và phép nhân list chép **tham chiếu** chứ không chép nội dung. Kết quả là $p[0], p[1], \dots$ cùng trỏ vào một hàng: ghi vào hàng 0 là ghi vào tất cả. C++ không có lỗi tương ứng, vì `vector<vector<ll>> p(n+1, vector<ll>(m+1))` chép thật.

Triệu chứng của lỗi này rất dễ đọc nhầm. Vòng dựng bảng vẫn chạy ra kết quả trông hợp lý, vì công thức truy hồi chỉ cần **hàng ngay trên**, mà hàng ngay trên lúc đó chính là hàng vừa ghi. Chỉ tới lúc truy vấn, khi cần $p[r1-1]$ với $r1-1$ cách xa dòng cuối, thì mọi thứ mới sai.

Cách viết nhanh trong Python. Đừng viết hai vòng lặp lồng nhau chạy qua 10^6 ô bằng trình thông dịch. Dựng từng hàng bằng `accumulate` cho tổng tiền tố theo hàng, rồi cộng cả hàng phía trên bằng một zip:

```
hang[1:] = [x + y for x, y in zip(accumulate(map(int, data[ptr:ptr + m])), tren[1:])]
```

Đo trên lưới 1000×1000 với $2 \cdot 10^5$ truy vấn, tức test lớn nhất của bài D:

Cách viết	Thời gian
accumulate cộng zip, bản trong solutions/ref	216 ms
hai vòng lặp Python chạy qua từng ô	277 ms
bản C++ của kho, cùng máy	812 ms

Bộ nhớ. Lưới 1000×1000 chứa 10^6 số. Trong C++ với long long là 8 MB. Trong Python, một list các số nguyên nhỏ chiếm khoảng 8 byte cho con trỏ cộng với đối tượng số nguyên được dùng chung, nên nó cùng cỡ ấy, nhưng đừng suy ra rằng Python luôn tốn như C++: số nguyên lớn hơn 2^{30} sẽ cấp phát đối tượng riêng. Bài này giới hạn $|a_{ij}| \leq 10^9$ và tổng tới 10^{15} , nên các ô của bảng tiền tố là đối tượng riêng thật.

4. Hai con trỏ

Mẫu hình chung: hai chỉ số cùng chạy về một hướng và **không bao giờ lùi lại**. Tổng số bước là $O(n)$ dù nhìn qua tưởng là hai vòng lặp lồng nhau. Cách đếm chi phí đúng không phải là đếm vòng lặp lồng nhau, mà là đếm tổng số lần một phần tử bị đụng tới.

Hai tình huống hay gặp:

Trên một dãy đã sắp. Tìm cặp có tổng bằng X: con trỏ trái ở đầu, phải ở cuối, tổng lớn quá thì lùi phải, nhỏ quá thì tiến trái.

Trên hai dãy đã sắp. Duyệt song song và mỗi bước tiến con trỏ đang chỉ vào giá trị **nhỏ hơn**. Đó là bài I: muốn tìm cặp (a_i, b_j) có chênh lệch nhỏ nhất thì chỉ cần so cặp hiện tại rồi tiến bên nhỏ hơn, vì tiến bên lớn hơn chắc chắn làm khoảng cách rộng ra.

Riêng với Python. Hàm sorted là Timsort viết bằng C, nên đừng tự viết sắp xếp. Đo trên test lớn nhất của bài I với $n = m = 2 \cdot 10^5$: đọc và tách 6 ms, ép kiểu và sắp cả hai dãy 46 ms, còn vòng hai con trỏ chạy bằng Python mất 57 ms. Tức là hai lần sắp xếp rẻ hơn chính cái vòng lặp $O(n)$ đi sau nó. Đó là cái tỷ lệ cần nhớ khi ước lượng thời gian một bài Python: phần chạy trong C gần như miễn phí, phần chạy trong trình thông dịch mới là phần bạn phải đếm.

5. Cửa sổ trượt và điều kiện đơn điệu

Cửa sổ trượt là hai con trỏ cùng chạy tiến, giữ một đoạn $[l, r]$ và một đại lượng tổng hợp của đoạn đó.

```
l = 0
for r in range(n):
    them(a[r]) # mở rộng bên phải
    while cua_so_khong_hop_le():
        bot(a[l]); l += 1 # thu hẹp bên trái
    tot_nhat = max(tot_nhat, r - l + 1)
```

Điều kiện để dùng được, và đây là phần quan trọng nhất của cả buổi. Cửa sổ trượt chỉ đúng khi tính chất bạn quan tâm có tính **đơn điệu**: nếu đoạn $[l, r]$ đã hợp lệ thì mọi đoạn con của nó cũng hợp lệ, hoặc ngược lại, mở rộng đoạn chỉ làm nó xấu đi theo một chiều duy nhất.

Với dãy toàn số dương, tổng đoạn tăng khi mở rộng và giảm khi thu hẹp. Đó chính là lý do bài E và bài F dùng được cửa sổ trượt. Số lượng giá trị phân biệt cũng vậy: thu hẹp đoạn thì số giá trị phân biệt không thể tăng, nên bài G dùng được.

Nếu tính chất không đơn điệu thì cửa sổ trượt **sai**, và đây là chỗ sinh viên gây nhiều nhất, vì code vẫn chạy và vẫn ra một con số.

Hai bài E và F khác nhau đúng một dòng, và đó là dòng đáng soi nhất.

Bài	Điều kiện thu hẹp	Hỏng thế nào nếu viết nhầm
E, đếm đoạn có tổng bằng S	while tong > S	viết >= thì vút luôn cửa sổ sắp được đếm, đáp án thành 0
F, đoạn ngắn nhất tổng >= S	while tong - a[l] >= S	viết while tong >= S thì thu hẹp quá tay, mất chính đáp án vừa tìm ra

Cả hai cách viết sai đều trông hợp lý khi đọc. Cách phát hiện chắc chắn nhất là viết một bộ vét cạn nhỏ rồi đối chiếu vài nghìn ca ngẫu nhiên với n từ 1 tới 8.

6. Khi hai con trỏ không dùng được

Bài H cổ tình giống bài E nhưng cho phép số âm. Chỉ một thay đổi đó thôi là toàn bộ lập luận đơn điệu sụp: thu hẹp cửa sổ có thể làm tổng **tăng lên**, nên khi thấy tổng lớn hơn mục tiêu bạn không có quyền kết luận là phải bỏ bớt bên trái.

Cách đúng là quay lại tổng tiền tố. Đoạn $a[l..r]$ có tổng bằng k khi và chỉ khi $p[r] - p[l-1] = k$, tức là $p[l-1] = p[r] - k$. Vậy chỉ cần đếm xem trong các tổng tiền tố đã đi qua có bao nhiêu giá trị bằng $p[r] - k$.

```
seen = {0: 1}          # tiền tố rỗng
p = 0
dap_an = 0
for x in a:
    p += x
    dap_an += seen.get(p - k, 0)
    seen[p] = seen.get(p, 0) + 1
```

Dòng khởi tạo bảng đếm là dòng quan trọng nhất. Viết `seen = {0: 1}`, và số 1 ấy đại diện cho tiền tố rỗng, và thiếu nó thì mọi đoạn bắt đầu từ vị trí 1 đều bị bỏ sót. Đây là lỗi cho ra kết quả gần đúng, nên test mẫu nhỏ vẫn có thể qua.

Thứ tự hai dòng trong thân vòng lặp cũng là một cái bẫy. Phải cộng đáp án **trước** rồi mới ghi tiền tố hiện tại vào bảng. Làm ngược lại thì với $k = 0$, tiền tố vừa ghi sẽ ghép với chính nó và bạn đếm thêm một đoạn rỗng ở mỗi vị trí.

Riêng với Python, và đây là chỗ đi ngược chiều với C++. Bản C++ viết `seen[p]++` và `std::map` tự tạo phần tử bằng 0 khi khóa chưa có. Python **không** làm thế: `seen[p] += 1` với khóa mới ném ngay `KeyError`. Ba cách đúng, và cách nào cũng được:

Cách	Ghi chú
<code>seen[p] = seen.get(p, 0) + 1</code>	không cần import gì, dùng ở bản mẫu
<code>collections.defaultdict(int)</code>	gần bản C++ nhất, nhưng đọc một khóa lạ cũng tạo ra nó
<code>collections.Counter</code>	tiện khi đếm cả dãy một lượt, thừa khi đếm dần

`dict` của Python là bảng băm nên tra cứu là $O(1)$ trung bình, còn `std::map` của C++ là cây cân bằng nên là $O(\log n)$. Vì vậy lời giải Python của bài G và bài H là $O(n)$ chứ không phải $O(n \log n)$, và đo được: bài G mất 59 ms bên này so với 113 ms bên C++.

Ghi nhớ cặp đối lập này, vì nó là bài học lớn nhất của buổi:

Dữ liệu	Kỹ thuật đúng
chỉ có số dương, hỏi đếm hoặc tìm đoạn theo tổng	hai con trỏ, $O(n)$
có số âm	tổng tiền tố kèm bảng đếm, $O(n)$ với <code>dict</code>

7. Hàng đợi đơn điệu

Bài toán: tìm giá trị lớn nhất của mọi cửa sổ độ dài k . Làm thẳng là $O(n \cdot k)$.

Ý tưởng: giữ một deque chứa **chỉ số**, sao cho giá trị tương ứng giảm dần từ đầu tới cuối. Khi phần tử mới tới, mọi phần tử nhỏ hơn hoặc bằng nó ở cuối hàng đợi bị loại vĩnh viễn, vì chúng vừa cũ hơn vừa nhỏ hơn nên không bao giờ còn cơ hội làm giá trị lớn nhất nữa.

```
from collections import deque

dq = deque() # chỉ số, giá trị giảm dần từ đầu tới cuối
for r in range(n):
    while dq and dq[0] <= r - k:
        dq.popleft() # đã rơi khỏi cửa sổ
    while dq and a[dq[-1]] <= a[r]:
        dq.pop() # bị che vĩnh viễn
    dq.append(r)
    if r >= k - 1:
        out.append(a[dq[0]])
```

Mỗi chỉ số vào hàng đợi đúng một lần và ra đúng một lần, nên tổng chi phí là $O(n)$ dù có hai vòng while lồng bên trong.

Luôn lưu chỉ số chứ đừng lưu giá trị. Vì bạn cần biết phần tử đầu hàng đợi đã rơi ra ngoài cửa sổ hay chưa, mà điều đó chỉ tra được từ chỉ số. Và điều kiện hết hạn là $dq[0] \leq r - k$, không phải $<$: chỉ số $r - k$ đã nằm ngoài cửa sổ $[r - k + 1, r]$ rồi.

Chọn deque hay list, và câu trả lời không phải cái bạn tưởng. Người ta hay dạy rằng `list.pop(0)` biến thuật toán này thành $O(n^2)$ nên chắc chắn quá thời gian. Tôi đã đo, và kết luận ấy **không đúng trên bộ test của kho này**:

Dữ liệu	deque	list với pop(0)
test 09 của kho, $n = 2 \cdot 10^5$, $k = 1000$	65 ms	65 ms
dãy giảm dần, $n = 2 \cdot 10^5$, $k = 10^5$	55 ms	1149 ms

Hai dòng ấy nói hai điều khác nhau, và cả hai đều cần nhớ. Điều thứ nhất: chi phí của `pop(0)` tỷ lệ với **độ dài hàng đợi**, mà với $k = 1000$ hàng đợi ngắn nên nó không lộ ra. Điều thứ hai: đề cho phép k tới n , nên lỗi ấy là lỗi thật, chỉ là bộ test hiện có không ép nó lộ ra. Một bộ test không bắt được một lỗi không có nghĩa lỗi đó không tồn tại, và đó là lý do phải dùng deque ngay cả khi list vừa cho AC.

8. Đo thật, mười bài cạnh nhau

Dưới đây là toàn bộ mười bài, trên test lớn nhất của từng bài, cùng máy, mỗi bên chạy ba lượt lấy lượt nhanh nhất, tính cả thời gian nạp trình thông dịch:

Bài	C++ của kho	Python của kho	Python trên C++
A, tổng đoạn	404 ms	87 ms	0,22
B, điểm cân bằng	67 ms	28 ms	0,42
C, mảng hiệu	424 ms	99 ms	0,23
D, tổng hai chiều	812 ms	230 ms	0,28
E, đếm đoạn tổng S	64 ms	31 ms	0,48
F, đoạn ngắn nhất	97 ms	49 ms	0,51
G, k giá trị phân biệt	113 ms	59 ms	0,52
H, có số âm	69 ms	35 ms	0,51
I, cặp gần nhau nhất	318 ms	96 ms	0,30
J, cửa sổ lớn nhất	197 ms	65 ms	0,33

Python nhanh hơn trên cả mười bài, từ 1,9 lần tới 4,6 lần. Đây không phải một lời khen Python, nó là một đặc điểm của buổi này: cả mười bài đều **nặng vào ra và nhẹ tính toán**. Bài nặng nhất, bài D, đọc 10^6 số rồi trả lời $2 \cdot 10^5$ truy vấn, mà mỗi truy vấn chỉ là bốn phép trừ cộng. Python đọc cả khối rồi tách một lần; còn `>>` của C++ trả giá cho từng số.

Đừng rút ra một tỷ lệ rồi nhân vào. Ở chủ đề 06 của chính kho này, bài dùng lazy propagation chạy 0,61 s bên C++ và 2,67 s bên Python, tức chậm 4,4 lần. Cùng một cặp ngôn ngữ, cùng một máy, hai chủ đề cho hai chiều ngược nhau. Cách duy nhất biết là đo, và đó là lý do nhánh này có `tools/qa-thoi-gian.py` đo từng bài trên từng test.

Hai điều phải nói rõ về bảng trên, để bạn biết nó đáng tin tới đâu. Thứ nhất, con số C++ đo trên bản dịch bằng `clang` của macOS với một file `bits/stdc++.h` tự dựng, vì `clang` không có sẵn header đó; máy khác với `g++` và `libstdc++` có thể ra số khác. Thứ hai, cả hai cột đều tính cả thời gian khởi động tiến trình, khoảng 15 ms cho Python và khoảng 2 ms cho C++, nên phần tính toán thuần của Python còn nhanh hơn tỷ lệ trong bảng một chút.

Giới hạn của buổi này là 1 giây cho mọi bài, đúng bằng bản C++, và riêng bài D thì chặt hơn bản C++ vốn cho 2 giây. Con số 1 giây đặt theo số đo: bài chậm nhất là bài D, đo nhiều lượt cho 216 tới 272 ms, tức khoảng một phần tư giới hạn, còn chín bài kia đều dưới 120 ms.

9. Đọc dữ liệu và ghi kết quả

Dòng code quan trọng nhất của cả buổi, và nó giống nhau ở cả mười bài:

```
data = sys.stdin.buffer.read().split()
```

Đọc hết đầu vào thành một khối byte rồi tách một lần. Sau đó đi bằng con trỏ `p` và ép kiểu khi cần: `int(data[p])`.

Đo trên bài A, test 09, file 4,7 MB với $n = q = 2 \cdot 10^5$:

Cách đọc	Thời gian
<code>sys.stdin.buffer.read().split()</code> một lần	82 ms
<code>sys.stdin.readline()</code> từng dòng	102 ms
<code>input()</code> từng dòng	200 ms

Chênh lệch giữa dòng đầu và dòng cuối là 2,4 lần. Với giới hạn 1 giây của buổi này thì cả ba đều lọt, nên xin nói thẳng: ở buổi này `input()` **không** làm bạn quá thời gian. Nó sẽ làm, ở những buổi mà phần tính toán đã ăn gần hết giới hạn. Học thói quen từ bây giờ rẻ hơn là sửa lúc đã TLE.

Về phía ghi, gom kết quả vào một list rồi ghi một lần bằng `'\n'.join`. Đo trên cùng bài A, bản gọi `print` từng dòng mất 93 ms so với 82 ms. Khoảng cách nhỏ vì `print` ghi vào một pipe đã có đệm; khi ghi ra màn hình thật thì nó lớn hơn nhiều.

Có một ngoại lệ phải biết: bài nào cần đọc **cả dòng kể cả dấu cách** thì không dùng được `split()`. Khi đó đọc bằng `sys.stdin.buffer.readline()` và cắt đúng ký tự xuống dòng bằng `rstrip("\n")`, chứ không phải `.strip()`, vì `.strip()` ăn mất dấu cách có nghĩa ở hai đầu. Buổi này không có bài nào như vậy, nhưng buổi 14 thì có.

10. Cạm bẫy riêng của Python

Cạm bẫy	Hậu quả
<code>p = [[0] * m] * n</code>	<code>n</code> hàng cùng trỏ vào một list, sửa hàng 0 là sửa tất cả
<code>d[i] += v</code> với <code>i</code> âm	không báo lỗi, ghi vào phần tử cuối mảng
<code>freq[x] += 1</code> với khóa mới	<code>KeyError</code> , khác hẳn <code>std::map</code> vốn tự tạo phần tử 0
<code>/</code> thay <code>//</code>	<code>/</code> luôn cho số thực, và số thực mất chính xác từ khoảng $9 \cdot 10^{15}$
ghép chuỗi bằng <code>s += ...</code> trong vòng lặp	$O(n^2)$ vì chuỗi Python bất biến, dùng list rồi <code>join</code>
<code>list.pop(0)</code> trong hàng đợi dài	tỷ lệ với độ dài hàng đợi, xem số đo ở mục 7
gọi <code>max(a, b)</code> trong vòng lặp nóng	lời gọi hàm đắt, viết <code>a if a > b else b</code>
để biến ở mức module thay vì trong <code>main()</code>	biến toàn cục tra theo từ điển, biến cục bộ tra theo chỉ số

Ba dòng cuối là chuyện của những bài nặng tính toán. Ở buổi này chúng gần như không đo được, vì không bài nào chạm tới một phần tư giới hạn. Đừng vì thế mà bỏ qua chúng: chúng là cùng một thói quen sẽ quyết định buổi 6 và buổi 12.

11. Những cạm bẫy của bản C++ đã biến mất

Bản C++ của buổi này liệt kê tám cạm bẫy. Ba trong số đó **không tồn tại** trong Python, và nói rõ điều này quan trọng ngang với dạy những cái còn lại, vì một danh sách chép nguyên từ ngôn ngữ khác sẽ dạy người học đề phòng những thứ không có thật.

Cạm bẫy của bản C++	Trạng thái bên Python
mảng tiền tố để <code>int</code> nên tổng tràn	biến mất, số nguyên Python không có tràn
biến đếm số đoạn để <code>int</code> , $2 \cdot 10^{10}$ vượt ngưỡng	biến mất, cùng lý do
hiệu hai số cỡ 10^9 trái dấu vượt <code>int</code> ở bài I	biến mất, cùng lý do
<code>d[r+1]</code> ghi ra ngoài mảng khi <code>r = n</code>	còn, nhưng đổi hình dạng: <code>IndexError</code> thay vì im lặng
lệch một đơn vị ở <code>p[l-1]</code>	còn nguyên, và nguy hiểm hơn vì <code>p[-1]</code> không báo lỗi
dùng cửa sổ trượt khi dữ liệu có số âm	còn nguyên, không dính gì tới ngôn ngữ
quên <code>seen[0] = 1</code>	còn nguyên
quên xóa khóa khỏi bảng đếm khi về 0	còn nguyên, nhưng triệu chứng thành <code>IndexError</code>
lưu giá trị thay vì chỉ số trong hàng đợi	còn nguyên

Và bản Python có thêm hai cạm bẫy mà bản C++ không có: dòng cấp phát bảng hai chiều ở mục 3, và `KeyError` khi tăng một khóa chưa tồn tại ở mục 6.

12. Bộ khung tự hỏi khi gặp đề mới

Thấy đề nói về đoạn con liên tiếp, hãy hỏi theo thứ tự này:

- Có truy vấn nhiều lần trên dãy không đổi không? Nếu có, nghĩ tổng tiền tố.

- Có cập nhật cả đoạn rồi mới hỏi ở cuối không? Nếu có, nghĩ mảng hiệu.
- Tính chất cần tìm có đơn điệu theo độ dài đoạn không? Nếu có, nghĩ cửa sổ trượt.
- Nếu không đơn điệu, dữ liệu có số âm không? Nếu có, nghĩ tổng tiền tố kèm bảng đếm.
- Cần cực trị của một cửa sổ độ dài cố định? Nghĩ hàng đợi đơn điệu.
- Ước lượng xem có bao nhiêu lượt vòng lặp chạy **trong trình thông dịch**, chứ không phải bao nhiêu phép tính. Phần chạy trong C, tức sorted, accumulate, sum, map, split, gần như miễn phí so với phần chạy trong Python.
- Dữ liệu vào lớn bao nhiêu byte? Trên 1 MB thì bắt buộc đọc cả khối một lần.

Phần II. Mười bài tập

Đề PDF	Ở đây	Kỹ thuật chính	File nộp
A	Bài 1	tổng tiền tố một chiều	bai01.py
B	Bài 2	tổng tiền tố, chia hai phía	bai02.py
C	Bài 3	mảng hiệu	bai03.py
D	Bài 4	tổng tiền tố hai chiều	bai04.py
E	Bài 5	hai con trỏ	bai05.py
F	Bài 6	cửa sổ trượt ngắn nhất	bai06.py
G	Bài 7	cửa sổ trượt kèm bảng tần suất	bai07.py
H	Bài 8	tổng tiền tố kèm bảng đếm	bai08.py
I	Bài 9	hai con trỏ trên hai dãy	bai09.py
J	Bài 10	hàng đợi đơn điệu	bai10.py

Giới hạn thời gian **1 giây** cho mọi bài, kể cả bài D vốn được bản C++ cho 2 giây. Con số này đặt theo số đo chứ không theo tỷ lệ; xem mục I.8. Dữ liệu vào lên tới 7,5 MB, nên đọc cả khối một lần là bắt buộc chứ không phải tùy chọn.

Tự chấm:

```
python3 chay.py topics/02-prefix-twopointer/solutions/bai01.py
python3 chay.py topics/02-prefix-twopointer --tat-ca
```

Bài A. Truy vấn tổng đoạn

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên và q truy vấn. Mỗi truy vấn cho hai chỉ số l và r , hãy tính tổng các phần tử từ vị trí l đến vị trí r .

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_1 tới a_n ($|a_i| \leq 10^9$)
- q dòng tiếp theo, mỗi dòng gồm l và r ($1 \leq l \leq r \leq n$)

Kết quả

q dòng, mỗi dòng là một tổng.

Ví dụ

Dữ liệu vào	Kết quả
5 3	15
1 2 3 4 5	5
1 5	4
2 3	
4 4	

Lưu ý

Trước khi nộp, hãy thử tay một truy vấn có $l = 1$. Đó là chỗ công thức hay lệch đi một đơn vị, và trong Python thì lệch một đơn vị về phía âm **không báo lỗi**: `p[-1]` lấy phần tử cuối mảng và trả về một con số trông rất bình thường.

Giải thích

Tổng cả dãy 1, 2, 3, 4, 5 là 15. Đoạn từ vị trí 2 tới 3 là $2 + 3 = 5$. Đoạn chỉ gồm vị trí 4 là chính giá trị 4.

Bài B. Điểm cân bằng

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên. Một vị trí i được gọi là điểm cân bằng nếu tổng các phần tử đứng trước i bằng tổng các phần tử đứng sau i . Bản thân a_i không thuộc bên nào cả. Hãy đếm số điểm cân bằng.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
6 1 7 3 6 5 6	1

Lưu ý

Nếu bạn thấy mình đang viết một vòng lặp bên trong một vòng lặp trên cùng dãy này, hãy dừng lại và đếm số phép tính. Phần tử a_i không thuộc bên nào, nên nhớ trừ nó ra khỏi vế phải.

Giải thích

Tại $i = 4$, tổng bên trái là $1 + 7 + 3 = 11$ và tổng bên phải là $5 + 6 = 11$. Không vị trí nào khác thỏa mãn, nên đáp án là 1.

Bài C. Cộng cả đoạn

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Ban đầu bạn có một mảng n phần tử, tất cả bằng 0. Có q thao tác, mỗi thao tác cho ba số l, r, v nghĩa là cộng thêm v vào mọi phần tử từ vị trí l đến vị trí r . Hãy in mảng sau khi thực hiện hết các thao tác.

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- q dòng tiếp theo, mỗi dòng gồm l, r, v ($1 \leq l \leq r \leq n, |v| \leq 10^9$)

Kết quả

Một dòng gồm n số, là mảng cuối cùng.

Ví dụ

Dữ liệu vào	Kết quả
5 3 1 3 2 2 5 -1 3 3 10	2 1 11 -1 -1

Lưu ý

Thực hiện thẳng từng thao tác là $O(n \cdot q)$, tức $4 \cdot 10^{10}$ phép tính. Hãy tìm cách để mỗi thao tác chỉ chạm vào hai ô nhớ. Khi đã tìm ra, hãy nghĩ xem mảng của bạn cần dài bao nhiêu khi r bằng n .

Giải thích

Lệnh thứ nhất cộng 2 vào các vị trí 1 tới 3, lệnh thứ hai cộng -1 vào các vị trí 2 tới 5, lệnh thứ ba cộng 10 vào riêng vị trí 3. Cộng dồn lại được dãy 2, 1, 11, -1, -1.

Bài D. Tổng hình chữ nhật

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho lưới n dòng và m cột. Có q truy vấn, mỗi truy vấn cho bốn số r_1 c_1 r_2 c_2 , hãy tính tổng mọi ô nằm trong hình chữ nhật có góc trên trái là (r_1, c_1) và góc dưới phải là (r_2, c_2) .

Dữ liệu vào

- Dòng 1: n, m, q ($1 \leq n, m \leq 1000, n \cdot m \leq 10^6, 1 \leq q \leq 2 \cdot 10^5$)
- n dòng tiếp theo, mỗi dòng m số nguyên ($|a_{ij}| \leq 10^9$)
- q dòng tiếp theo, mỗi dòng gồm r_1 c_1 r_2 c_2 ($1 \leq r_1 \leq r_2 \leq n, 1 \leq c_1 \leq c_2 \leq m$)

Kết quả

q dòng, mỗi dòng là một tổng.

Ví dụ

Dữ liệu vào	Kết quả
3 3 2	12
1 2 3	28
4 5 6	
7 8 9	
1 1 2 2	
2 2 3 3	

Lưu ý

Đây là bài nặng nhất của buổi, dữ liệu vào 7,5 MB. Hãy vẽ bốn hình chữ nhật ra giấy trước khi viết công thức, đừng học thuộc dấu cộng trừ. Và hãy soi kỹ dòng bạn cấp phát mảng hai chiều: trong Python có đúng một cách viết sai làm mọi hàng trở chung vào một chỗ.

Giải thích

Hình chữ nhật thứ nhất gồm bốn ô 1, 2, 4, 5 nên tổng bằng 12. Hình thứ hai gồm 5, 6, 8, 9 nên tổng bằng 28.

Bài E. Đếm đoạn con có tổng đúng bằng S

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên **dương** và một số S . Hãy đếm số đoạn con liên tiếp có tổng đúng bằng S .

Dữ liệu vào

- Dòng 1: n và S ($1 \leq n \leq 2 \cdot 10^5$, $1 \leq S \leq 10^{14}$)
- Dòng 2: n số nguyên a_i ($1 \leq a_i \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
6 6 2 3 1 4 2 2	2

Lưu ý

Ràng buộc mọi phần tử đều dương không phải để trang trí. Hãy tự hỏi nó cho bạn tính chất gì về tổng của một đoạn khi bạn mở rộng hoặc thu hẹp đoạn đó. Điều kiện thu hẹp cửa sổ là dấu lớn hơn hay lớn hơn hoặc bằng, và tại sao?

Giải thích

Hai đoạn có tổng bằng 6 là 2 3 1 và 4 2, nên đáp án là 2.

Bài F. Đoạn con ngắn nhất có tổng không nhỏ hơn S

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên **dương** và một số S . Hãy tìm độ dài nhỏ nhất của một đoạn con liên tiếp có tổng lớn hơn hoặc bằng S . Nếu không có đoạn nào, in 0 .

Dữ liệu vào

- Dòng 1: n và S ($1 \leq n \leq 2 \cdot 10^5, 1 \leq S \leq 10^{14}$)
- Dòng 2: n số nguyên a_i ($1 \leq a_i \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
6 7 2 3 1 2 4 3	2

Lưu ý

Điều kiện thu hẹp cửa sổ phải là bỏ phần tử bên trái đi mà cửa sổ **vẫn còn** hợp lệ. Viết nhầm thành thu hẹp chừng nào đang hợp lệ sẽ làm mất chính đáp án vừa tìm được. Trường hợp không có đoạn nào phải in 0 , đừng in giá trị khởi tạo của biến kết quả, và trong Python thì `float('inf')` in ra chữ `inf` chứ không phải một số.

Giải thích

Đoạn `4 3` có tổng `7` và độ dài `2`, và không có đoạn nào ngắn hơn đạt tổng `7`.

Bài G. Đoạn con nhiều nhất k giá trị phân biệt

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên và một số k . Hãy tìm độ dài lớn nhất của một đoạn con liên tiếp chứa không quá k giá trị phân biệt.

Dữ liệu vào

- Dòng 1: n và k ($1 \leq k \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($1 \leq a_i \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
7 2 1 2 1 3 4 3 3	4

Lưu ý

Khi số đếm của một giá trị tụt về 0, hãy nghĩ kỹ xem khóa đó còn nên nằm trong bảng tần suất nữa hay không. Ngoài ra, tăng số đếm của một khóa chưa có trong dict sẽ ném `KeyError`, khác hẳn `std::map` của C++ vốn tự tạo phần tử bằng 0.

Giải thích

Đoạn 3 4 3 3 chỉ chứa hai giá trị phân biệt và dài 4, không đoạn nào dài hơn mà vẫn giữ được không quá hai giá trị.

Bài H. Đếm đoạn con có tổng bằng k

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên, lần này **có thể âm**, và một số k . Hãy đếm số đoạn con liên tiếp có tổng đúng bằng k .

Dữ liệu vào

- Dòng 1: n và k ($1 \leq n \leq 2 \cdot 10^5$, $|k| \leq 10^{14}$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
5 0 1 -1 2 -2 3	3

Lưu ý

Hãy thử áp lời giải của bài E vào dãy trong ví dụ và chạy tay. Chỗ nó hỏng chính là nội dung mà bài này muốn dạy. Có hai chi tiết nhỏ quyết định đáp án: tiền tố rỗng có mặt trong bảng đếm ngay từ đầu hay không, và trong thân vòng lặp thì cộng đáp án trước hay ghi bảng trước.

Giải thích

Ba đoạn có tổng bằng 0 là 1 -1, 2 -2 và 1 -1 2 -2, nên đáp án là 3.

Bài I. Cặp gần nhau nhất giữa hai dãy

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho hai dãy số nguyên, dãy thứ nhất có n phần tử và dãy thứ hai có m phần tử. Hãy tìm giá trị nhỏ nhất của $|a_i - b_j|$ trên mọi cặp.

Dữ liệu vào

- Dòng 1: n và m ($1 \leq n, m \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)
- Dòng 3: m số nguyên b_j ($|b_j| \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
4 3 1 5 9 14 4 8 20	1

Lưu ý

Thử mọi cặp là $4 \cdot 10^{10}$ phép. Sau khi sắp cả hai dãy, hãy tự trả lời: đang đứng ở một cặp, tiến con trỏ nào thì khoảng cách mới có cơ hội nhỏ đi? Tiến nhằm bên vẫn ra một con số, và con số đó đúng đủ thường xuyên trên test nhỏ để bạn tin nó.

Giải thích

Cặp (5, 4) có chênh lệch 1, và không cặp nào có chênh lệch nhỏ hơn.

Bài J. Giá trị lớn nhất của mọi cửa sổ

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên và một số k . Với mỗi đoạn con liên tiếp có độ dài đúng bằng k , hãy in ra giá trị lớn nhất của đoạn đó, theo thứ tự từ trái sang phải.

Dữ liệu vào

- Dòng 1: n và k ($1 \leq k \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)

Kết quả

Một dòng gồm $n - k + 1$ số.

Ví dụ

Dữ liệu vào	Kết quả
8 3 1 3 -1 -3 5 3 6 7	3 3 5 5 6 7

Lưu ý

Lời giải $O(n \cdot k)$ sẽ quá thời gian. Câu hỏi mở đường: nếu có hai vị trí $i < j$ mà $a_i \leq a_j$, thì a_i còn cơ hội làm giá trị lớn nhất của một cửa sổ chứa cả hai nữa không? Khi đã có cấu trúc rồi, hãy dùng `collections.deque` chứ đừng dùng `list`, và mục I.7 có số đo giải thích vì sao.

Giải thích

Sáu cửa sổ lần lượt là 1 3 -1, 3 -1 -3, -1 -3 5, -3 5 3, 5 3 6 và 3 6 7, với giá trị lớn nhất lần lượt là 3, 3, 5, 5, 6, 7.

Phần III. Gợi ý và lời giải

Đọc từng tầng một, dừng lại ngay khi đã nghĩ ra. Code đầy đủ nằm ở Phần IV.

Bài	Cái bẫy thật sự	Riêng với Python
A	lệch một đơn vị ở $p[l-1]$	$p[-1]$ không báo lỗi, nó trả phần tử cuối
B	tự tính lại tổng phải mỗi bước thành $O(n^2)$	không có
C	mảng hiệu thiếu ô khi $r = n$	thành <code>IndexError</code> chứ không im lặng
D	đầu của bốn số hạng trong công thức bù trừ	<code>[[0]*m]*n</code> làm mọi hàng trở chung một chỗ
E	thu hẹp cửa sổ khi tổng bằng 5	không có
F	thu hẹp cửa sổ sai thời điểm nên bỏ sót đáp án	<code>float('inf')</code> in ra chữ, không phải số
G	quên xóa khóa khỏi bảng khi đếm về 0	<code>freq[x] += 1</code> với khóa mới ném <code>KeyError</code>
H	quên tiền tố rỗng, và sai thứ tự hai dòng	nghư bài G
I	tiến nhằm con trỏ	<code>sorted</code> là hàm C, đừng tự viết sắp xếp
J	lưu giá trị thay vì chỉ số trong hàng đợi	<code>list.pop(0)</code> tỷ lệ với độ dài hàng đợi

Bài A. Truy vấn tổng đoạn

Gợi ý 1. Dãy không đổi trong suốt quá trình truy vấn. Bạn được phép tiền xử lý một lần.

Gợi ý 2. Nếu biết trước tổng của i phần tử đầu tiên với mọi i , thì tổng một đoạn tính bằng một phép trừ.

Gợi ý 3. $p[i] = p[i-1] + a[i]$, và đáp án là $p[r] - p[l-1]$.

Lời giải. Xây bảng $O(n)$, mỗi truy vấn $O(1)$, tổng $O(n + q)$. Mảng p dài $n + 1$ với $p[0] = 0$, và $p[1:] = \text{accumulate}(\dots)$ điền phần còn lại bằng một vòng chạy trong C .

Lệch một đơn vị, và vì sao nó nguy hiểm hơn ở Python. Viết $p[r] - p[l]$ thay vì $p[r] - p[l-1]$ là lỗi phổ biến nhất của cả buổi. Trong C++ nếu l bằng 0 thì $p[l-1]$ là truy cập ngoài vùng và có cơ hội nổ ra. Trong Python $p[-1]$ là phần tử **cuối** mảng, hoàn toàn hợp lệ, và bạn nhận về tổng cả dãy thay vì một lỗi. Cách chống dứt điểm vẫn như cũ: đánh số mảng từ 1, khai p dài $n + 1$, và luôn thử tay một truy vấn có $l = 1$ trước khi nộp.

Không phải lo. Tổng đạt $2 \cdot 10^{14}$, bên C++ đó là lý do phải dùng `long long`. Số nguyên Python không tràn nên bài này mất đúng cái bẫy ấy.

Python. Đây là bài chứng minh rõ nhất của mục I.9. Cùng thuật toán, chỉ đổi cách đọc: một khối 82 ms, readline từng dòng 102 ms, `input()` từng dòng 200 ms. Bản C++ của kho mất 404 ms trên cùng test, vì cịn >> trả giá cho từng số trong khi `split()` cắt cả khối bằng một lượt C.

Bài B. Điểm cân bằng

Gợi ý 1. Với mỗi i , bạn cần hai đại lượng: tổng bên trái và tổng bên phải. Đừng tính lại từ đầu ở mỗi bước.

Gợi ý 2. Nếu biết tổng cả dãy, tổng bên phải suy ra được từ tổng bên trái mà không cần cộng thêm gì.

Gợi ý 3. `phai = tong - trai - a[i]`.

Lời giải. Một lượt quét, giữ biến `tra` cộng dồn. Độ phức tạp $O(n)$, không cần cả mảng tiền tố.

Cái bẫy là bậc hai mà không nhận ra. Nhiều người viết vòng lặp trong để tính tổng bên phải ở mỗi vị trí. Với n bằng $2 \cdot 10^5$ thì đó là $4 \cdot 10^{10}$ phép. Hề thấy mình viết vòng lặp lồng nhau trên cùng một dãy, hãy dừng lại và hỏi xem có thể cộng dồn được không.

Cái bẫy thứ hai, nhỏ hơn nhưng dễ mắc hơn. Phần tử `a_i` không thuộc bên nào, nên về phải là `tong - tra` - `a[i]` chứ không phải `tong - tra`. Bỏ quên số hạng cuối cho đáp án 0 trên ví dụ mẫu, tức là lỗi lộ ra ngay, nhưng chỉ khi bạn thật sự chạy ví dụ mẫu.

Python. `sum(a)` là một vòng chạy trong `C` nên tổng cả dãy gần như miễn phí, không cần gộp nó vào vòng quét chính cho gọn.

Bài C. Cộng cả đoạn

Gợi ý 1. Làm thẳng là $O(n \cdot q)$. Hãy đếm thử: $4 \cdot 10^{10}$ phép.

Gợi ý 2. Một thao tác cộng vào cả đoạn làm thay đổi **hiệu giữa hai phần tử liền kề** ở đúng hai chỗ: mép trái và ngay sau mép phải.

Gợi ý 3. Ghi `d[l] += v` và `d[r+1] -= v`, cuối cùng lấy tổng tiền tố của `d`.

Lời giải. Đây là mảng hiệu, còn gọi là kỹ thuật imos. Mỗi thao tác $O(1)$, khôi phục mảng $O(n)$, tổng $O(n + q)$.

Khai mảng hiệu dài $n + 2$. Khi `r = n` thì `d[r+1]` là `d[n+1]`. Trong C++ đây là lỗi ghi ra ngoài vector và nó không báo gì cả. Trong Python cùng lỗi ấy ném `IndexError` ngay dòng đó, nên bộ chấm kết luận RE và bạn biết ngay. Đừng vì thế mà nghĩ Python luôn che lưng cho bạn: nếu lỗi đi về phía chỉ số âm thì nó im lặng hoàn toàn.

Không phải lo. Giá trị cuối có thể tới $2 \cdot 10^{14}$, bên C++ phải khai `long long`, bên này không.

Python. Khôi phục mảng bằng `accumulate(d[1:n+1])` rồi `' '.join` một lần. Kỹ thuật này chỉ áp dụng khi **mọi cập nhật đến trước mọi truy vấn**. Nếu đề xen kẽ cập nhật và truy vấn thì phải dùng Fenwick tree, là nội dung buổi sau.

Bài D. Tổng hình chữ nhật

Gợi ý 1. Đây là bài A nhưng thêm một chiều. Hãy định nghĩa `p[i][j]` là tổng của cả hình chữ nhật từ góc $(1,1)$ tới (i,j) .

Gợi ý 2. Khi xây `p[i][j]`, hai hình chữ nhật `p[i-1][j]` và `p[i][j-1]` chồng lên nhau ở đâu?

Gợi ý 3. Chồng nhau ở `p[i-1][j-1]`, nên phần đó bị cộng hai lần và phải trừ ra một lần. Ở công thức truy vấn thì ngược dấu.

Lời giải. Hai công thức nằm ở mục I.3. Chúng có dấu **ngược nhau** ở số hạng góc, và đó chính là chỗ hay nhớ nhầm. Cách chắc chắn nhất là vẽ bốn hình chữ nhật ra giấy chứ đừng học thuộc.

Cái bẫy riêng của Python nằm ở dòng cấp phát. `[[0] * (m+1)] * (n+1)` tạo $n+1$ tham chiếu tới **một** hàng, vì phép nhân list chép tham chiếu chứ không chép nội dung. Phải viết `[[0] * (m+1) for _ in range(n+1)]`. Điều làm lỗi này sống dai: vòng dựng bảng vẫn cho số đúng, vì truy hồi chỉ đọc hàng ngay trên, mà hàng ngay trên lúc đó chính là hàng vừa ghi. Chỉ tới lúc truy vấn cần `p[r-1]` ở xa thì đáp án mới sai.

Python. Đừng chạy hai vòng lặp lồng nhau qua 10^6 ô. Dựng từng hàng bằng `accumulate` rồi cộng hàng trên bằng `zip`: đo được 216 ms so với 277 ms, và bản C++ của kho mất 812 ms. Đây là bài nặng nhất của buổi, đo được 216 tới 272 ms tùy lượt, trên tổng giới hạn 1000 ms.

Bài E. Đếm đoạn con có tổng đúng bằng S

Gợi ý 1. Đề nhấn mạnh mọi phần tử đều dương. Điều đó cho bạn tính chất gì về tổng của một đoạn khi mở rộng hoặc thu hẹp?

Gợi ý 2. Tổng tăng thật sự khi mở rộng và giảm thật sự khi thu hẹp. Nghĩa là khi tổng đã vượt S , mở rộng thêm chẳng ích gì.

Gợi ý 3. Hai con trỏ cùng tiến, không bao giờ lùi.

Lời giải. Mở rộng bên phải, thu hẹp bên trái **chừng nào tổng còn lớn hơn S** , rồi kiểm tra bằng đúng S . Độ phức tạp $O(n)$.

Dấu lớn hơn, không phải lớn hơn hoặc bằng. Viết `while tong >= S` thì mỗi lần cửa sổ vừa chạm đúng S bạn lại phá nó đi trước khi kịp đếm, và chương trình in ra 0 trên mọi dữ liệu. Đây là một trong hai chỗ mà bài E và bài F khác nhau, và chúng khác nhau đúng chỗ ấy.

Vì sao vòng while bên trong không làm thuật toán thành bậc hai. Vì l chỉ tiến chứ không bao giờ lùi, nên tổng số lần thân vòng `while` chạy trong cả chương trình nhiều nhất là n .

Python. Cách dùng tổng tiền tố kèm bảng đếm cũng đúng và tổng quát hơn, và với `dict` thì nó cũng là $O(n)$ chứ không chậm hơn một hệ số log như bản C++ với `std::map`. Mục đích của bài này vẫn là để bạn nhận ra khi nào **không cần** tối nó.

Bài F. Đoạn con ngắn nhất có tổng không nhỏ hơn S

Gợi ý 1. Cùng một cửa sổ trượt như bài E, chỉ khác điều kiện dừng.

Gợi ý 2. Khi cửa sổ đã hợp lệ, bạn muốn nó **hợp nhất có thể** mà vẫn hợp lệ.

Gợi ý 3. `while l <= r and tong - a[l] >= S: tong -= a[l]; l += 1`, rồi mới cập nhật đáp án.

Lời giải. Điều kiện thu hẹp phải là "bỏ `a[l]` đi mà vẫn còn hợp lệ" chứ không phải "đang hợp lệ". Viết `while tong >= S` là sai, vì bạn sẽ thu hẹp quá tay tới lúc cửa sổ không còn hợp lệ và làm mất chính cái đáp án vừa tìm được. Trên ví dụ mẫu, bản sai in ra 0.

Đây là kiểu lỗi rất khó thấy khi đọc code, vì cả hai cách viết đều trông hợp lý. Cách phát hiện chắc chắn nhất là stress test với n từ 1 tới 8 so với vết cạn.

Giá trị khởi tạo. Trường hợp không có đoạn nào phải in 0, đừng in giá trị khởi tạo của biến kết quả.

Python. Bản C++ dùng `INT_MAX` làm mốc "chưa tìm thấy". Python không có hằng ấy. Hai lựa chọn: `float('inf')`, nhưng nhớ rằng in nó ra sẽ cho chữ `inf` chứ không phải một số, hoặc `n + 1`, vốn lớn hơn mọi bề rộng thật và vẫn là số nguyên. Bản mẫu chọn cách thứ hai.

Bài G. Đoạn con nhiều nhất k giá trị phân biệt

Gợi ý 1. Nếu một đoạn có không quá k giá trị phân biệt thì mọi đoạn con của nó cũng vậy. Tính chất đó gọi là đơn điệu, và nó cho phép dùng cửa sổ trượt.

Gợi ý 2. Cửa sổ cần biết số giá trị phân biệt hiện có, tức là cần một bảng tần suất.

Gợi ý 3. Khi số khóa của bảng vượt k , đẩy mép trái sang phải tới khi không vượt nữa.

Lời giải. Một `dict` đếm tần suất, và `len(freq)` chính là số giá trị phân biệt. Độ phức tạp $O(n)$, vì `dict` là bảng băm.

Dòng xóa khóa là dòng quyết định. Nó là `del freq[y]`. Nếu bạn chỉ giảm đếm về 0 mà không xóa khóa, khóa đó vẫn nằm trong `dict` và `len(freq)` vẫn tính nó là một giá trị phân biệt. Cửa sổ không bao giờ co đủ, vòng `while` không thoát, và con trỏ trái chạy quá mãng.

Cùng một lỗi, hai hình dạng triệu chứng. Bên C++ nó hiện ra thành đáp án nhỏ hơn thực tế, vì `freq.size()` chỉ đếm sai chữ vector vẫn cho đọc ngoài vùng mà không kêu. Bên Python nó hiện ra thành `IndexError`, vì `a[l]` với l bằng n là lỗi thật. Đây là một ví dụ tốt để nói với sinh viên: cùng một lỗi có thể lộ ra dưới nhiều hình dạng khác nhau, và hình dạng dễ thấy hơn không có nghĩa là lỗi nhẹ hơn.

Python. `freq[x] += 1` với một khóa chưa có ném `KeyError`, khác hẳn `std::map` vốn tự tạo phần tử bằng 0. Dùng `freq.get(x, 0) + 1`, hoặc `collections.defaultdict(int)`, hoặc `collections.Counter`. Đo được: 59 ms bên này so với 113 ms bên C++, và phần chênh chủ yếu là `dict` băm so với `map` cây.

Bài H. Đếm đoạn con có tổng bằng k

Gợi ý 1. Thử áp cửa sổ trượt của bài E vào dãy 1 -1 2 -2 3 và tự chạy tay. Nó hỏng ở đâu?

Gợi ý 2. Với số âm, thu hẹp cửa sổ có thể làm tổng **tăng lên**. Toàn bộ lập luận đơn điệu sụp, nên hai con trỏ không dùng được.

Gợi ý 3. Quay lại định nghĩa: tổng đoạn $[l, r]$ bằng k nghĩa là $p[l-1] = p[r] - k$. Vậy chỉ cần đếm xem đã gặp giá trị đó bao nhiêu lần.

Lời giải. Một dict đếm số lần mỗi giá trị tổng tiền tố xuất hiện, khởi tạo bằng $\{0: 1\}$. Với dict thì độ phức tạp là $O(n)$.

Ba cái bẫy.

- **Thiếu tiền tố rỗng.** Tiền tố rỗng phải được đếm, nếu không thì mọi đoạn bắt đầu từ vị trí đầu tiên đều bị bỏ sót. Trên ví dụ mẫu, bản thiếu nó in ra 1 thay vì 3. Kết quả gần đúng nên test nhỏ vẫn có thể qua.
- **Thứ tự trong vòng lặp.** Phải **cộng đáp án trước** rồi mới ghi `seen[p]`, nếu không thì với $k = 0$ bạn đếm cả đoạn rỗng ở mỗi vị trí.
- **Biến đếm để kiểu int** là cái bẫy của bản C++, vì số đoạn con tối đa gần $2 \cdot 10^{10}$. Bên Python nó biến mất.

Ghi nhớ cặp bài E và H như một cặp: cùng một câu hỏi, chỉ khác một dòng ràng buộc, mà lời giải đối hần. Khi đọc đề, dòng nói về dấu của dữ liệu quan trọng ngang với dòng nói về kích thước.

Python. Cùng chuyện `KeyError` như bài G. Và đừng dùng `seen.keys()` hay `in seen` rồi mới lấy giá trị: `seen.get(p - k, 0)` chỉ bấm một lần thay vì hai.

Bài I. Cặp gần nhau nhất giữa hai dãy

Gợi ý 1. Thử mọi cặp là $O(n \cdot m)$, tức $4 \cdot 10^{10}$. Không được.

Gợi ý 2. Nếu cả hai dãy đã sắp xếp, và bạn đang đứng ở $a[i]$ với $b[j]$, thì việc tiến con trỏ nào có khả năng làm khoảng cách nhỏ đi?

Gợi ý 3. Chỉ có tiến con trỏ đang chỉ vào giá trị **nhỏ hơn**. Tiến bên lớn hơn chắc chắn làm khoảng cách rộng ra.

Lời giải. Sắp cả hai rồi đi hai con trỏ song song. Độ phức tạp $O(n \log n + m \log m)$, phần chi phối trên giấy là hai lần sắp xếp.

Nhưng trên máy thì không. Đo trên test lớn nhất với $n = m = 2 \cdot 10^5$: đọc và tách 6 ms, ép kiểu và sắp cả hai dãy 46 ms, vòng hai con trỏ 57 ms. Hai lần sắp xếp $O(n \log n)$ chạy trong C rẻ hơn một vòng $O(n)$ chạy trong trình thông dịch. Đây là bài học về cách ước lượng thời gian của một chương trình Python: đừng đếm phép tính, hãy đếm **số lượt vòng lặp chạy trong trình thông dịch**.

Tiến nhằm con trỏ. Đổi chiều so sánh vẫn cho ra một con số, và trên ví dụ mẫu nó in 3 thay vì 1. Trên dữ liệu ngẫu nhiên lớn thì nó đúng khá thường xuyên, vì hai dãy dày đặc nên gần như chỗ nào cũng có cặp sát nhau. Đó là kiểu lỗi mà chỉ test nhỏ được thiết kế riêng mới bắt được.

Python. `sorted` là Timsort viết bằng C, đừng tự viết sắp xếp. Và `abs()` là một lời gọi hàm, nên trong vòng nóng bản mẫu trừ tay theo hai nhánh thay vì gọi nó.

Bài J. Giá trị lớn nhất của mọi cửa sổ

Gợi ý 1. Khi cửa sổ trượt sang phải một bước, một phần tử vào và một phần tử ra. Bạn cần một cấu trúc trả lời được giá trị lớn nhất mà không phải quét lại cả cửa sổ.

Gợi ý 2. Nếu có hai phần tử $i < j$ mà $a[i] \leq a[j]$, thì $a[i]$ có bao giờ còn là giá trị lớn nhất của một cửa sổ chứa cả hai không?

Gợi ý 3. Không bao giờ, vì $a[j]$ vừa lớn hơn hoặc bằng lại vừa sống lâu hơn. Vậy hãy loại $a[i]$ ngay khi $a[j]$ xuất hiện. Cái còn lại là một dãy giảm dần, giữ trong deque.

Lời giải. Mỗi chỉ số vào một lần và ra một lần, nên $O(n)$ dù có hai vòng `while` bên trong. Đây là cùng một lỗi phân tích chi phí như ở bài E: đừng đếm vòng lặp lồng nhau, hãy đếm tổng số lần một phần tử bị đụng tới.

Ba điểm phải nhớ.

- **Lưu chỉ số chứ không lưu giá trị.** Bạn cần biết phần tử đầu hàng đợi đã rơi khỏi cửa sổ hay chưa, và chỉ chỉ số mới nói được điều đó.
- **Điều kiện hết hạn dùng dấu nhỏ hơn hoặc bằng.** Phải là $dq[0] \leq r - k$, không phải $dq[0] < r - k$, vì chỉ số $r - k$ đã nằm ngoài cửa sổ $[r - k + 1, r]$. Viết nhầm dấu thì phần tử hết hạn ở lại thêm đúng một bước, và lỗi chỉ lộ ra khi phần tử ấy đang là giá trị lớn nhất, tức là trên dãy giảm dần.
- **Thứ tự hai vòng while.** Loại phần tử hết hạn ở đầu trước, rồi mới loại phần tử bị che ở cuối.

Dùng $\leq$ khi loại ở cuối để hàng đợi không phình ra vì các giá trị bằng nhau. Dùng $<$ vẫn cho kết quả đúng nhưng tốn bộ nhớ hơn, nên đó là chuyện tối ưu chứ không phải chuyện đúng sai.

Python, và đây là chỗ tôi phải đính chính một lời dạy quen thuộc. Người ta hay nói `list.pop(0)` biến bài này thành $O(n^2)$ nên chắc chắn TLE. Tôi đã đo trên bộ test của kho này: deque 65 ms, list với `pop(0)` cũng 65 ms, không phân biệt nổi. Lý do là test lớn nhất dùng $k = 1000$ nên hàng đợi luôn ngắn. Trên một dãy giảm dần với $k = 10^5$, tức đúng loại dữ liệu ép hàng đợi dài, hai con số là 55 ms và 1149 ms. Kết luận đúng là: chi phí của `pop(0)` tỷ lệ với **độ dài hàng đợi**, để cho phép k tới n , nên phải dùng deque; nhưng bộ test hiện có **không** chứng minh được điều đó, và nói rằng nó chứng minh là nói quá.

Đổi `max` thành `min` thì chỉ cần đảo chiều so sánh. Cấu trúc này còn là nền của tối ưu quy hoạch động bằng hàng đợi đơn điệu ở các buổi sau.

Phần IV. Mười code mẫu đầy đủ

Mười file này là **đúng** nội dung trong solutions/ref/, tức đúng thứ bộ chấm chạy. Công tools/qa-note-code.py so từng dòng và trượt nếu hai bên lệch nhau, vì mã in ra giấy không được bộ test nào chấm nên nó là chỗ dễ lạc hậu nhất trong cả kho.

Bài A. Truy vấn tổng đoạn

```
# Problem A: range sum queries answered from a prefix sum table.
# p[i] is the sum of the first i elements, so the sum of a[l..r] with 1 based indices
# is p[r] - p[l-1]. Building p costs O(n) and every query costs O(1).
#
# The trap. Writing p[r] - p[l] instead of p[r] - p[l-1] is the most common mistake of
# the whole session. Keeping p one slot longer and indexing the array from 1 means the
# index l-1 is never negative, so the formula needs no special case.
#
# Python note. The C++ version must declare p as long long, because the sum reaches
# 2*10^14 and int stops at about 2*10^9. Python integers grow on demand, so that entire
# trap does not exist here. What replaces it is the cost of input. Measured on this
# repo's largest test for this problem, 4.7 MB with n = q = 2*10^5:
#   read().split() once, the shape below ..... 82 ms
#   sys.stdin.readline() per line ..... 102 ms
#   input() per line ..... 200 ms
# The same test takes 404 ms with the C++ reference of this repo, because cin >> pays
# per token while split() cuts the whole buffer in one C level pass.
import sys
from itertools import accumulate

def main():
    data = sys.stdin.buffer.read().split()
    p = 0
    n = int(data[p]); q = int(data[p + 1]); p += 2

    # One extra slot at the front, so p[0] = 0 and the index l-1 never goes negative.
    tien_to = [0] * (n + 1)
    tien_to[1:] = accumulate(map(int, data[p:p + n]))
    p += n

    out = []
    for _ in range(q):
        l = int(data[p]); r = int(data[p + 1]); p += 2
        out.append(tien_to[r] - tien_to[l - 1])

    sys.stdout.write('\n'.join(map(str, out)) + '\n')

main()
```

Bài B. Điểm cân bằng

```
# Problem B: count the positions whose left part and right part have equal sums.
# One pass with a running prefix is enough. At position i the left sum is the running
# total of everything before i, and the right sum is total - left - a[i], because a[i]
# itself belongs to neither side.
#
```

```

# The trap. Recomputing the right sum with an inner loop at every position turns this
# into  $O(n^2)$ , which is  $4 \cdot 10^{10}$  operations at  $n = 2 \cdot 10^5$ . Whenever an inner loop walks
# the same array again, stop and ask whether a running total would do.
#
# Python note. sum() is a C level loop over the list, so the grand total is computed
# before the scan rather than maintained alongside it. No overflow guard is needed:
# the C++ version declares the running sums as long long for a total near  $2 \cdot 10^{14}$ .
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    a = list(map(int, data[1:1 + n]))

    tong = sum(a)
    trai = 0
    dem = 0
    for x in a:
        # The element x sits on neither side, so it is removed from the right part.
        phai = tong - trai - x
        if trai == phai:
            dem += 1
        trai += x

    sys.stdout.write(str(dem) + '\n')

main()

```

Bài C. Cộng cả đoạn

```

# Problem C: apply  $q$  range additions to an array of zeros, then print the array.
# The difference array turns one range addition into two point updates:  $d[l] += v$  marks
# the start and  $d[r+1] -= v$  cancels it just past the end. One prefix sum at the very end
# rebuilds the array. Cost  $O(n + q)$  instead of  $O(n \cdot q)$ .
#
# The trap. The array must have  $n + 2$  slots, because when  $r$  equals  $n$  the index  $r + 1$  is
#  $n + 1$ . In C++ that write lands outside the vector and nothing complains; in Python the
# same mistake raises IndexError, so the failure is loud instead of silent. Either way
# the size  $n + 2$  is the fix, not a try block.
#
# Python note. The rebuild is itertools.accumulate over the slice d[1..n], which runs in
#  $O(n)$ , and the answer is joined once instead of printed number by number.
import sys
from itertools import accumulate

def main():
    data = sys.stdin.buffer.read().split()
    p = 0
    n = int(data[p]); q = int(data[p + 1]); p += 2

    # Size  $n + 2$  so that  $d[r + 1]$  is always a real slot, even when  $r$  equals  $n$ .
    d = [0] * (n + 2)
    for _ in range(q):
        l = int(data[p]); r = int(data[p + 1]); v = int(data[p + 2]); p += 3
        d[l] += v
        d[r + 1] -= v

```

```
sys.stdout.write(' '.join(map(str, accumulate(d[1:n + 1]))) + '\n')

main()
```

Bài D. Tổng hình chữ nhật

```
# Problem D: rectangle sum queries on a grid, using a two dimensional prefix sum.
# p[i][j] is the sum of the sub grid from (1,1) to (i,j). Both formulas below are
# inclusion and exclusion, with the corner term carrying opposite signs:
# build  p[i][j] = a[i][j] + p[i-1][j] + p[i][j-1] - p[i-1][j-1]
# query  s = p[r2][c2] - p[r1-1][c2] - p[r2][c1-1] + p[r1-1][c1-1]
# Building is O(n*m) and every query is O(1).
#
# The Python trap of this problem is the allocation on the line marked below. Writing
# [[0] * (m + 1)] * (n + 1) gives n + 1 references to ONE row, so every write lands in
# every row. The comprehension form builds n + 1 separate rows. C++ has no counterpart
# to this mistake, which is why it is worth its own line of attention.
#
# The second Python point is speed. One row of the table is built with accumulate plus
# a zip, both C level loops, instead of a per cell Python loop over 10^6 cells.
# Measured on the 1000 x 1000 grid with 2*10^5 queries, the largest test here:
# accumulate plus zip, the shape below ..... 216 ms
# a Python loop over every cell ..... 277 ms
# the C++ reference of this repo ..... 812 ms
import sys
from itertools import accumulate

def main():
    data = sys.stdin.buffer.read().split()
    ptr = 0
    n = int(data[ptr]); m = int(data[ptr + 1]); q = int(data[ptr + 2]); ptr += 3

    # n + 1 SEPARATE rows. The multiplication form would alias them all together.
    p = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        hang = p[i]
        tren = p[i - 1]
        # Row prefix of the input row, plus the whole table above it, cell by cell.
        hang[1:] = [x + y for x, y in
                    zip(accumulate(map(int, data[ptr:ptr + m])), tren[1:])]
        ptr += m

    out = []
    for _ in range(q):
        r1 = int(data[ptr]); c1 = int(data[ptr + 1])
        r2 = int(data[ptr + 2]); c2 = int(data[ptr + 3]); ptr += 4
        duoi = p[r2]
        tren = p[r1 - 1]
        out.append(duoi[c2] - tren[c2] - duoi[c1 - 1] + tren[c1 - 1])

    sys.stdout.write('\n'.join(map(str, out)) + '\n')

main()
```

Bài E. Đếm đoạn con có tổng đúng bằng S

```
# Problem E: count the subarrays whose sum is exactly S, every element positive.
# Because every element is at least 1, the window sum strictly grows when the right end
# advances and strictly drops when the left end advances. That monotonicity is what lets
# two pointers work: the left pointer never has to go back, so the whole scan is O(n).
#
# The trap. The shrinking condition is "while the sum is GREATER than S", not "while it
# is at least S". Shrinking on equality throws away the very window that was going to be
# counted, and the program then answers 0 on every test.
#
# Python note. There is no overflow to worry about here, so S may reach 10^14 without a
# single cast, while the C++ version needs long long for both S and the window sum.
# Everything lives inside main(), so the interpreter reaches these names by slot number
# instead of by dictionary lookup, which is why this repo's template insists on it.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); S = int(data[1])
    a = list(map(int, data[2:2 + n]))

    tong = 0
    dem = 0
    l = 0
    for x in a:
        tong += x
        # Shrink only while the window is too big. Stopping at equality is the point.
        while tong > S:
            tong -= a[l]; l += 1
        if tong == S:
            dem += 1

    sys.stdout.write(str(dem) + '\n')

main()
```

Bài F. Đoạn con ngắn nhất có tổng không nhỏ hơn S

```
# Problem F: shortest subarray whose sum is at least S, every element positive.
# Same sliding window as problem E, one different question, and one different shrinking
# rule. The window is shrunk while dropping the leftmost element STILL leaves it valid,
# and only then is its width compared with the best seen so far.
#
# The trap. Writing "while the sum is at least S, shrink" looks just as reasonable and
# is wrong: it keeps shrinking one step too far, the window ends up invalid, and the
# answer that was sitting right there is lost. On the sample this prints 0.
#
# Python note. There is no INT_MAX here. The sentinel for "no window found yet" is
# n + 1, which is larger than any real width, and the final line turns it into 0. Using
# float('inf') would work too but then the printed value would be a float.
import sys

def main():
    data = sys.stdin.buffer.read().split()
```

```

n = int(data[0]); S = int(data[1])
a = list(map(int, data[2:2 + n]))

tong = 0
tot_nhat = n + 1          # larger than any real width, so it means "not found yet"
l = 0
for r in range(n):
    tong += a[r]
    # Drop the left element only while the window survives without it.
    while l <= r and tong - a[l] >= S:
        tong -= a[l]; l += 1
    if tong >= S:
        rong = r - l + 1
        if rong < tot_nhat:
            tot_nhat = rong

sys.stdout.write(str(0 if tot_nhat == n + 1 else tot_nhat) + '\n')

main()

```

Bài G. Đoạn con nhiều nhất k giá trị phân biệt

```

# Problem G: longest subarray containing at most k distinct values.
# Shrinking a window can never raise its number of distinct values, so the property is
# monotone and a sliding window applies. The window carries a frequency table; whenever
# the table holds more than k keys, the left end advances until it does not.
#
# The trap. When a count falls to zero the KEY MUST LEAVE the table. A leftover zero
# entry still counts towards len(freq), so the window keeps shrinking for a value that
# already left it. In C++ the symptom is a wrong, smaller answer; in Python the left
# pointer runs past the end of the list and the program dies with IndexError. Same bug,
# louder symptom.
#
# Python note. dict is a hash table with O(1) average lookup, so this is O(n) here while
# the C++ version with std::map is O(n log n); on the largest test that shows up as
# 59 ms here against 113 ms there. There is a second difference that bites:
# freq[x] += 1 on a missing key raises KeyError, because a Python dict does not create
# an entry on read the way std::map does. Use .get with a default, as below.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); k = int(data[1])
    a = list(map(int, data[2:2 + n]))

    freq = {}
    tot_nhat = 0
    l = 0
    for r in range(n):
        x = a[r]
        freq[x] = freq.get(x, 0) + 1
        while len(freq) > k:
            y = a[l]
            c = freq[y] - 1
            if c:
                freq[y] = c
            else:
                # Remove the key, do not just set it to zero.
                del freq[y]
            l += 1
        tot_nhat = max(tot_nhat, r - l + 1)

    sys.stdout.write(str(tot_nhat) + '\n')

```

```

        del freq[y]
        l += 1
        rong = r - l + 1
        if rong > tot_nhat:
            tot_nhat = rong

    sys.stdout.write(str(tot_nhat) + '\n')

main()

```

Bài H. Đếm đoạn con có tổng bằng k

```

# Problem H: count the subarrays whose sum is exactly k, negative values allowed.
# Two pointers do NOT work here. With negative values the window sum is no longer
# monotone, so a sum above the target says nothing about which end to move. Go back to
# the definition instead:  $\text{sum}(l..r) == k$  is the same as  $p[l-1] == p[r] - k$ , so the job
# is to count how many earlier prefixes carry that value.
#
# Two traps, and both leave a plausible looking number behind.
# 1. The empty prefix must be in the table from the start, otherwise every subarray
#    that begins at position 1 is missed. On the sample that gives 1 instead of 3.
# 2. The answer is added BEFORE the current prefix is stored. Storing first would let
#    a prefix pair with itself, which counts an empty subarray whenever k is 0.
#
# Python note. The C++ version needs long long for the counter, since the number of
# subarrays reaches  $2 \cdot 10^{10}$ ; Python integers make that trap disappear. What remains is
# the dict habit: seen[p] += 1 raises KeyError on a new key, so .get with a default is
# the shape to learn.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); k = int(data[1])

    seen = {0: 1}          # the empty prefix, so subarrays starting at index 1 count
    p = 0
    dap_an = 0
    for tu in data[2:2 + n]:
        p += int(tu)
        # Count first, store second: with k = 0 the other order counts the empty piece.
        dap_an += seen.get(p - k, 0)
        seen[p] = seen.get(p, 0) + 1

    sys.stdout.write(str(dap_an) + '\n')

main()

```

Bài I. Cặp gần nhau nhất giữa hai dãy

```

# Problem I: smallest absolute difference between an element of a and an element of b.
# Trying every pair is  $4 \cdot 10^{10}$  operations. Sort both sequences, then walk one pointer
# through each: compare the current pair, then advance whichever side holds the SMALLER
# value, because moving the larger side can only widen the gap.

```

```
#
# The trap. Advancing the wrong side still produces a number, and that number is right
# often enough on tiny tests to look convincing. On the sample it prints 3 instead of 1.
#
# Python note. The C++ version needs long long, because the difference of two values
# near  $10^9$  with opposite signs passes the range of int; here that concern is gone.
# Measured on the largest test,  $n = m = 2 \cdot 10^5$ : reading and splitting 6 ms, converting
# and sorting both sequences 46 ms, and the two pointer walk 57 ms. So the two sorts,
# which run in C, cost less than the Python level loop that follows them.
# abs() is a function call, so the two branches subtract by hand instead.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); m = int(data[1])
    a = sorted(map(int, data[2:2 + n]))
    b = sorted(map(int, data[2 + n:2 + n + m]))

    tot_nhat = a[0] - b[0]
    if tot_nhat < 0:
        tot_nhat = -tot_nhat
    i = 0
    j = 0
    while i < n and j < m:
        x = a[i]; y = b[j]
        # Only advancing the smaller side can shrink the gap.
        if x < y:
            d = y - x
            if d < tot_nhat:
                tot_nhat = d
            i += 1
        else:
            d = x - y
            if d < tot_nhat:
                tot_nhat = d
            j += 1

    sys.stdout.write(str(tot_nhat) + '\n')

main()
```

Bài J. Giá trị lớn nhất của mọi cửa sổ

```
# Problem J: maximum of every window of width k.
# A monotonic deque holds INDICES whose values decrease from front to back, so the front
# is always the maximum of the current window. When a new element arrives, every element
# at the back that is not larger than it is dropped forever: it is both older and no
# bigger, so it can never be the maximum of a window that contains the new one. Each
# index is pushed once and popped once, hence  $O(n)$  despite the two inner while loops.
#
# Two traps.
# 1. Store indices, not values. Only an index tells whether the front has fallen out
# of the window, and the expiry test is  $\text{front} \leq r - k$ , not  $\text{front} < r - k$ .
# 2. Drop the expired front BEFORE dropping shadowed elements at the back.
#
# Python note, and it is a measured one rather than a guess. collections.deque pops
# from either end in  $O(1)$ ; a plain list is correct too, but  $\text{list.pop}(0)$  shifts every
# remaining element. On THIS repo's largest test the two are indistinguishable, 65 ms
```

```
# each, because that test uses k = 1000 so the queue never grows long. On a decreasing
# sequence with n = 2*10^5 and k = 10^5, where the queue really does hold k indices,
# the same code takes 55 ms with deque and 1149 ms with list.pop(0). A cost that the
# available test set cannot see is still a cost.
```

```
import sys
from collections import deque

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); k = int(data[1])
    a = list(map(int, data[2:2 + n]))

    dq = deque()          # indices, values decreasing from front to back
    out = []
    ghi = out.append
    for r in range(n):
        x = a[r]
        # Drop indices that have fallen out of the window on the left.
        while dq and dq[0] <= r - k:
            dq.popleft()
        # Drop smaller values at the back: a[r] shadows them from now on.
        while dq and a[dq[-1]] <= x:
            dq.pop()
        dq.append(r)
        if r >= k - 1:
            ghi(a[dq[0]])

    sys.stdout.write(' '.join(map(str, out)) + '\n')

main()
```